# UCF Physics AST4762/5765: Introduction to Numerical Computing
## Organizing your homework, homework format and coding style

*The following are written assuming that AST4762 as the class you take. If you are in the AST5765 section change the name accordingly*.

An important part of this class is to learn how to effectively structure your work on your computer. For this reason, we will follow a few rules that you need to abide by, and which are part of your homework grade. To start with, you need to choose a username that you will use throughout this class.

**Choosing a usernam**e: You need to pick and stick with a username, which becomes part of the filenames you hand in. For example, if your name is "Susan Sarah Smith", your username might be ss, sss, s3, sue, ssmith, sssmith, suess, susan, smith, etc. Keep it professional, which means name-based. Save the "interesting" names for your social life.

**Class directory**: Every file concerning this class (books, Jupyter notebooks, homework etc) should be inside one folder named `ast4762`. We will make this folder together in the class, and we will put it under revision control (we will explain what this is in the class).

**Class directory structure**: In the class folder you will make a number of subfolders:

a. `Homework/` this is the folder where you will have all the homework you work on. In this folder you will have a number of subfolders for each and every homework (see 'Homework submission' for their naming convention)
b. `Handin/` this is the folder where you will have the final, clean version of the homework folders that you will submit to Webcourses. Since you are handing things in for grading, you should keep separate copies of exactly what you handed in, so there is no question in your mind, or ours, in case of a grading dispute. This is called an "archive", and we use them a lot in research. The dates and other file metadata are important in archives! Never edit in an archive! Only copy files out of them or read them. Generally, original data, reports you publish or prepare for your supervisor or funding agency, reports you receive from your employees (someday!), etc. are all archival.
c. `Literature/` this is the folder where you will have all literature material.
d. `Lectures/` this is the folder where you will have all the pdfs and notebooks that we will use in the class.

A part of this class is learning how to use revision control. We will practice it by putting your `ast4762/` under revision control, and by using GitHub classroom. Through the latter you will be able to upload anything you want the TA or me to see and help you debug remotely (we will talk about this in the class).

**Homework naming conventions**: For each homework assignment you should make a new folder. Your folder should be named after the homework you are submitting, as `hw<number>_<username>`. For example, the folder of homework 1 of user ssmith will be named as: `hw1_ssmith`. In there you will have:

1. the main homework file (typically, this will be a .py file). This file should follow a similar naming convention indicating the number of the homework and your name. For example, the first code you will hand in will be in homework 2. User ssmith will name the Jupyter notebook: `hw2_ssmith.py`.

2. Any support files needed.
   2.1. The homework might need you to write functions that you then import in the main code. These functions will need to be stored in one or more separate files. These files will be non-executable Python files (`.py`). Your support files should be named in a way that clearly indicates what its contents are. For example, if you are asked to save all support functions of homework 5 in a single file, ssmith could name this file as: `hw5_ssmith_support_functions.py`. If you are asked to save a function named miles_to_km in a separate file, ssmith could name this file simply as `miles_to_km.py`.
   2.2. The homework might ask you to make a plot, a screenshot, or other graphics. These should be named in a way that clearly indicates which problem they are associated with. For example, in homework 2 ssmith needs to make a screenshot of the Jupyter notebook for problem 1, question a, and another screenshot of the GitHub desktop app for question b. The screenshots will then be named: `hw2_ssmith_problem1_graph1.png` and `hw2_ssmith_problem1_graph2.png`.
   2.3. In some homework you might need to save data that your code produces. These should be stored in a file that, again, indicates which problem they are associated with. For example, for homework 10 ssmith needs to store the output of a function for problem 3. This data file will be named as: `hw10_ssmith_problem3_data.txt` (or .dat).
   2.4. Last but not least, for some homework (and your Midterm and Final projects) you will need to read in data that we give you. You should keep this data in the folder without changing their name.

   **It is important that you copy these files in your `handin/` version of the homework folder as well**. We grade your homework by running your notebook. If a support file is missing your code will crash, and you will miss points!

3. The log file. Your log should be a simple text file named `0-ast4762-<username>.log`. For example, the log file of ssmith will be: `0-ast4762-ssmith.log`. This file should be in the main `ast4762/` folder, and you will update its contents every week. We will discuss how you will do this in class. Once you are done with the homework (e.g., `hw2/` ) and are about to submit your homework in

Webcourses, you will make a copy of the log file in the `handin/hw2/` so that the most up-to-date log file is submitted with your homework.

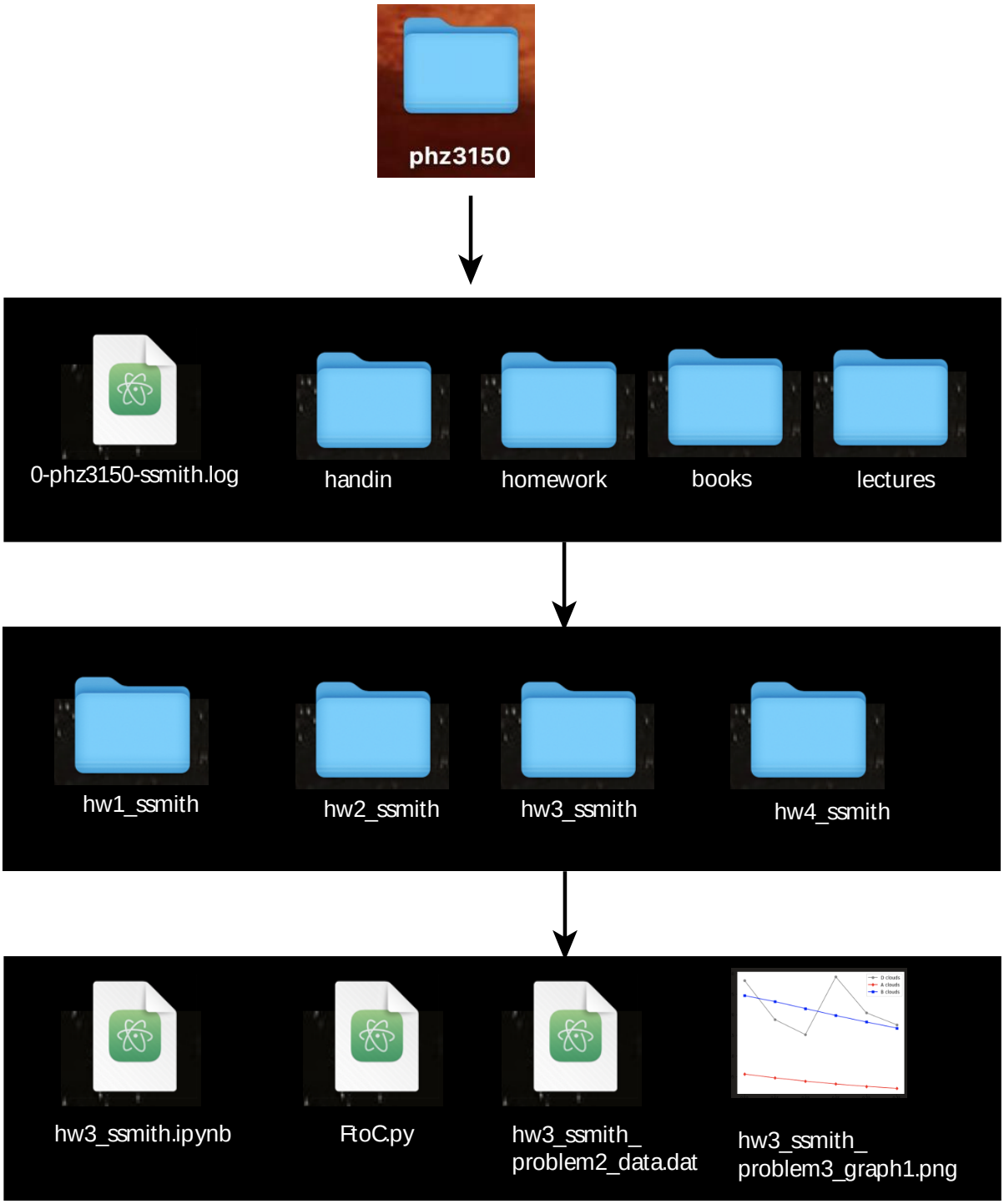



*Figure 1. A schematic of the phz3150/ structure, which follows the same the naming conventions we will use in this class. Notice the log file is at the 1st level of your folder.*

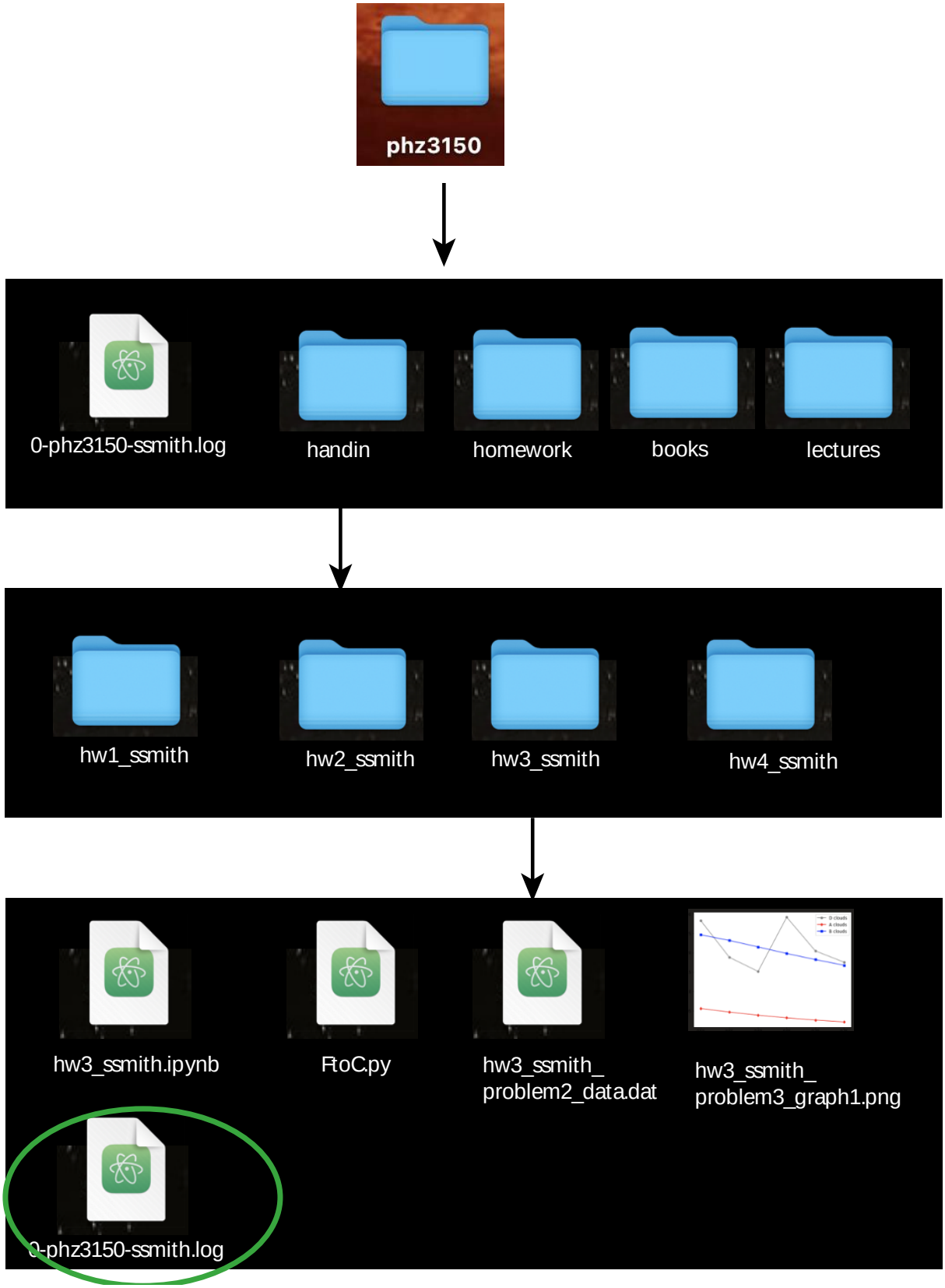



*Figure 2. (cont. Fig 1) Notice that the handin/ version of the hw3_ssmith/ folder is identical to that of the hw3_ssmith/ folder (Fig. 1) ) BUT it also contains a copy of the log file as it is at time of submission to Webcourses.*

**Homework submission**: All your homework should be submitted as single tar.gz files on Webcourses. Once you are satisfied with your homework, you will copy all files into the corresponding `handin/` folder. You then copy the log file in the folder (there will be reminders about this in each homework). Then use the tar.gz command we will learn in class. Submit the tar.gz on Webcourses.

**Information about the homework content:**

1. **Main homework file:** Your main homework file should start with information about: your name, the class name and the homework assignment number. These will appear as comments at the start of the notebook/.py file. Then, before the start of each problem you need to print the problem number. An example is shown here and we will remind you about the use of # and print() in class.

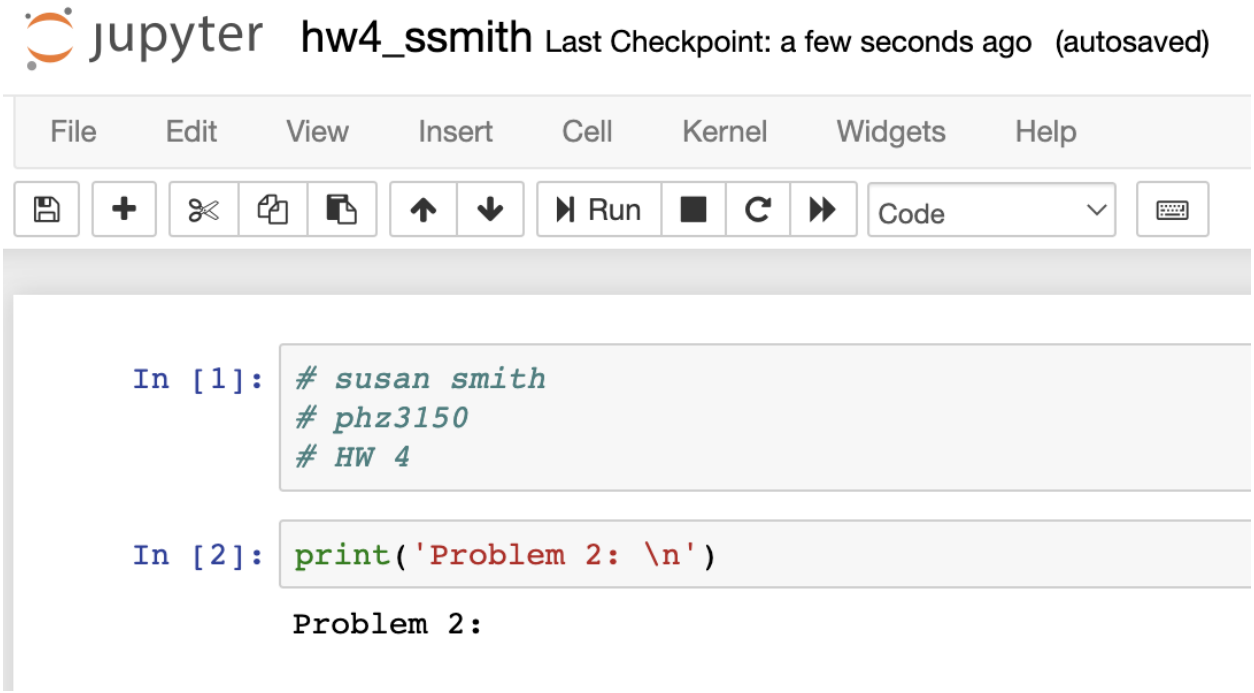



*Figure 3 An example of the information your homework should have at the start of the notebook.*

If asked to calculate a value as part of your homework you should print the value. This information should also be printed out in your log (see log).

Later in this class we will read data, import functions etc in the main Jupyter notebook. You should make sure that the paths you use are self-contained in the homework folder. If the paths point to a folder in your computer, your code will crash when we run it and cost you grading points. We will explain what all these means in the class.

2. **Functions:** All functions *should include docstrings*. Docstrings are informative pieces of text that let the user of your function know what the function does and what its inputs and outputs are. We will discuss in class how to write a docstring, and what information it should have.

3. **Log:** We will discuss the dos and don'ts of keeping a good log in the class. You can also find all information in Webcourses under Files/handouts/worklog_instructions.pdf. Your log should contain all information about what you do to solve the homework problems. If part of the problem asks you to calculate a value, make notes about your result in the log. This ensures that you will get partial credit, even if your program does not run for us.

**Coding style**

1. All integer-value floating-point numbers should have decimal points after them. For example, if you have a time of 10 sec, do not use `time = 10`, use `time = 10.` instead. An item count is always an integer (`i = 1, 2, 3`, …), but a distance is always a float (`dis = 1., 2., 3.`, …). A decimal in the range (-1,1) must always have a zero before the decimal point, for readability:

```
x = 0.23   # Right!
x = .23    # Wrong!
```

The purpose of this one is simply to build the decimal-point habit. In Python it's less of an issue now, but sometimes code is translated, and integer division is still out there.

2. Use spaces around binary operations and relations.

```
x = 5. + 1. # Right
x=5.+1.     # Wrong
```

This makes reading the code easier.

3. Keep lines to 80 characters, max, except in rare cases that are well justified, such as very long strings. If you make comments on the same line as code, keep them short or break them over more than a line:

--- instead of using this:

```
Value1 = ( ( x + y * z ) * 24 + 36 ) * 22 - 3
```

```
# Longer comment requiring much more space because
# I'm explaining something complicated.
```

--- you should use this:

```
Value1 = x + y * z        # set code Valuel to x + y * z
Value1 = Value1 * 24 + 36 # add this
Value1 = Value1 * 22 - 3  # then fix the whole expression.

# Longer comment requiring much more
# space because I'm explaining
# something complicated.
```

4. Keep blocks of similar lines internally lined up on decimals, comments, and = signs. This makes them easier to read and verify. There will be some cases when this is impractical. Use your judgment (you're not a computer, you control the computer!):

```
x    =   1.      # this is a comment
y    = 378.2345  # here's another
fred = chuck     # note how the decimals, = signs, and
                 # comments line up nicely...

alacazamshmazooboloid = 2721 # but not always!
```

5. Put the units and sources of values in comments:

```
t_planet = 523. # K, Smith and Jones (2016, ApJ 234, 22)
```

6. Align similar, adjacent code lines to make differences pop out and reduce the likelihood of bugs. For example, it is much easier to verify the correctness of:

```
a         = 3 * x + 3 * 8. * short1        - 5. * np.exp(np.pi..
a_altname = 3 * x + 3 * 8. * shortothernum - 5. * np.exp(np.pi.
```

than:

```
a = 3 * x + 3 * 8. * short1 - 5. * np.exp(np.pi * omega * t)
a_altname = 3 * x + 3*9* shortothernum - 5. * np.exp(np.pi * \
omega*t)
```

7. Use meaningful names for your variables, and use them in your formulae and functions:

```
time     = 5.           # s
distance = 12.          # m
```

```
<lots of other code here>

speed    = distance/time # m/s
```

Otherwise, later on when you read the code –or a client/colleague that gets to work with your code-- and update it,  might not remember which is the time variable and which the distance, and might calculate the 1/speed instead of the speed.

8. Similarly, it is very important that you use comments in your code. Say where information comes from, give information about algorithms chosen, the logical flow, and any coding choices you made. This will help you (or your client/colleague) a lot in the future when you revisit the code and want to do an update. We will talk about this in the class as well.

9. It's best if variables that will appear in formulae are short, so more terms can fit in one 80 character line.

Overall, having and following a style makes code easier to read. And, as an added bonus, if you take care to be consistent, you will write slower, view your code more times, and catch more bugs as you write them. Thus, for codes of any significant size, writing pedantically commented and aligned code is almost always faster than blast coding, if you include debugging time.